Algorithms, The Growth of Functions, Complexity of Algorithms

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Introduction to Algorithms
2. Common Algorithms
3. Growth of Functions
4. Big-O Notation
5. Big-Omega and Big-Theta
6. Time Complexity of Algorithms
7. Time Complexity Examples
8. Summary

What Is an Algorithm?

Motivating problems

In discrete mathematics and computer science, many questions look like:

- Given a list of integers, find the largest one.
- Given a set of integers, sort them in increasing order.
- Given a network, find the shortest path between two vertices.

To solve such problems systematically, we need a precise procedure.

Idea

We first model the problem using discrete structures: sets, sequences, functions, graphs, networks, and so on. Then we design a step-by-step method that always leads from the input to the desired output.

Definition

An **algorithm** is a finite sequence of precise instructions for performing a computation or for solving a problem.

Example: Largest Element in a Sequence

Problem

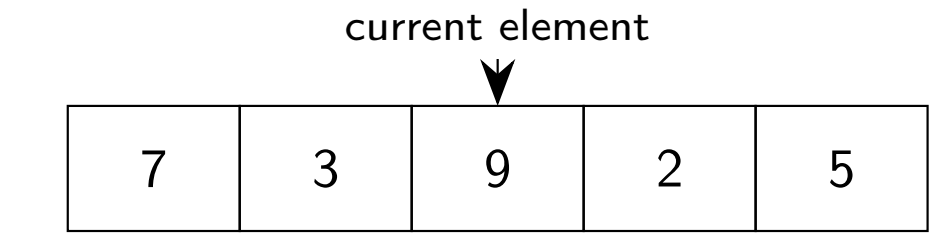
Describe an algorithm to find the maximum (largest) value in a finite sequence of integers

$$a_1, a_2, \ldots, a_n.$$

High-level steps

1. Set a *temporary maximum* equal to the first element a_1.
2. Move through the sequence one element at a time.
3. Whenever you see an element larger than the temporary maximum, update the temporary maximum.
4. When no elements remain to be checked, the temporary maximum is the largest integer in the sequence.

Example: Largest Element in a Sequence



start: max = 7 update: max = 9

Key idea

We keep **one** variable (`max`) and scan the sequence once. At each step we compare the corresponding with (`max`) and decide whether to change or keep it.

Algorithm 1: Maximum of a Finite Sequence

Pseudocode

procedure Max($a_1, a_2, \ldots, a_n$: integers)
  max := a_1
  for $i := 2$ **to** n **do**
    if max $< a_i$ **then** max := a_i
  return max {max is the largest element}

Observation

- The algorithm inspects each input exactly once.
- At any time, `max` is the largest value seen so far.

Common Properties of Algorithms

Desirable properties

Most algorithms we design and analyse have the following features:

- **Input:** values are taken from a specified input set.
- **Output:** at least one value is returned; this is the solution.
- **Definiteness:** every step is clear and unambiguous.
- **Correctness:** the algorithm gives the right output for every valid input.
- **Finiteness:** for every input, the algorithm terminates after a finite number of steps.
- **Effectiveness:** each step is simple enough to be carried out exactly in a finite amount of time.
- **Generality:** the same procedure works for all allowed inputs in the problem class, not just for one example.

The Searching Problem

General question

Given a list of distinct elements

$$a_1, a_2, \ldots, a_n$$

and a target value x, determine:

- the position i such that $a_i = x$, if it exists; or
- report that x is not in the list.

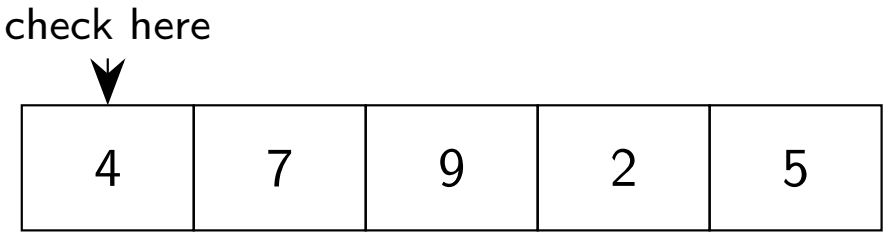
Strategy

- Start at the first element of the list.
- Compare it with the target x.
- If they are equal, we have found the location.
- If not, move one position to the right and repeat.
- Stop when x is found or the end of the list is reached.

Algorithm 2: Linear Search

Pseudocode

procedure LinearSearch(x : integer; $a_1, \ldots, a_n$: distinct integers)
  $i := 1$
  while $(i \leq n$ **and** $x \neq a_i)$ **do**
    $i := i + 1$
  if $i \leq n$ **then** location $:= i$
  else location $:= 0$
  return location {0 means "not found"}



move left-to-right until x is found

Note

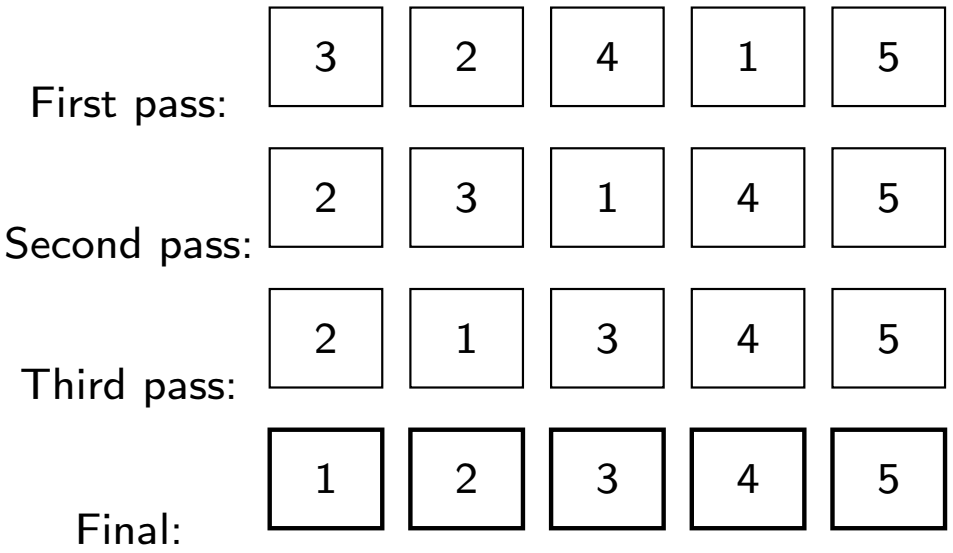
In the worst case, every element must be inspected once.

Motivation for Sorting

Bubble sort (informal idea)

Bubble sort repeatedly compares **adjacent** elements and interchanges them when they are in the wrong order. Large elements gradually **bubble up** toward the end of the list.

We sort the list $(3, 2, 4, 1, 5)$ into increasing order.



Observation

After each pass, the largest unsorted element has moved to its correct position at the right end of the unsorted part.

Algorithm 3: Bubble Sort

Pseudocode

procedure BubbleSort($a_1, \ldots, a_n$: real numbers with $n \geq 2$)
  for $i := 1$ **to** $n - 1$ **do**
    for $j := 1$ **to** $n - i$ **do**
      if $a_j > a_{j+1}$ **then interchange** a_j **and** a_{j+1}
  {after the loops, $a_1, \ldots, a_n$ are in increasing order}

Comment

Bubble sort is simple and good for teaching, but it is not very efficient for large lists compared to more advanced sorting algorithms.

What is Growth of Functions?

Idea

The **growth of a function** describes how fast its value increases when the input gets large.

In algorithm analysis

For an algorithm, we often get a function $T(n)$ that counts the number of steps for input size n. We care about:

- how quickly $T(n)$ grows as n gets large;
- comparing two algorithms: which one grows slower?

We usually ignore small details (like constant factors) and focus on the **overall growth rate**.

Growth of Functions: Intuition

Highest-order term dominates

For large x, the term with the highest power of x controls how fast a polynomial grows.

$$f(x) = x^2 + 1, \quad g(x) = x^2 + 2, \quad h(x) = x^2 + x + 1$$

All three grow at roughly the same rate as x^2 when x is large.

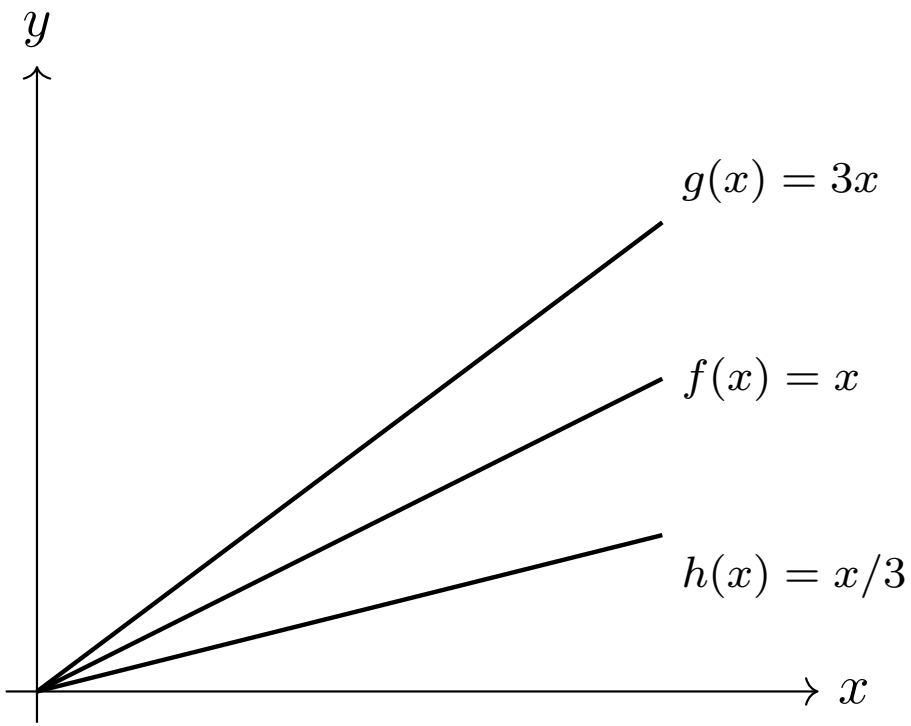
Constant multiples matter less

Similarly, for large x,

$$f(x) = x, \quad g(x) = 3x, \quad h(x) = \frac{x}{3}$$

all grow like a straight line. Their exact slope is different, but the **type of growth** is the same (linear).

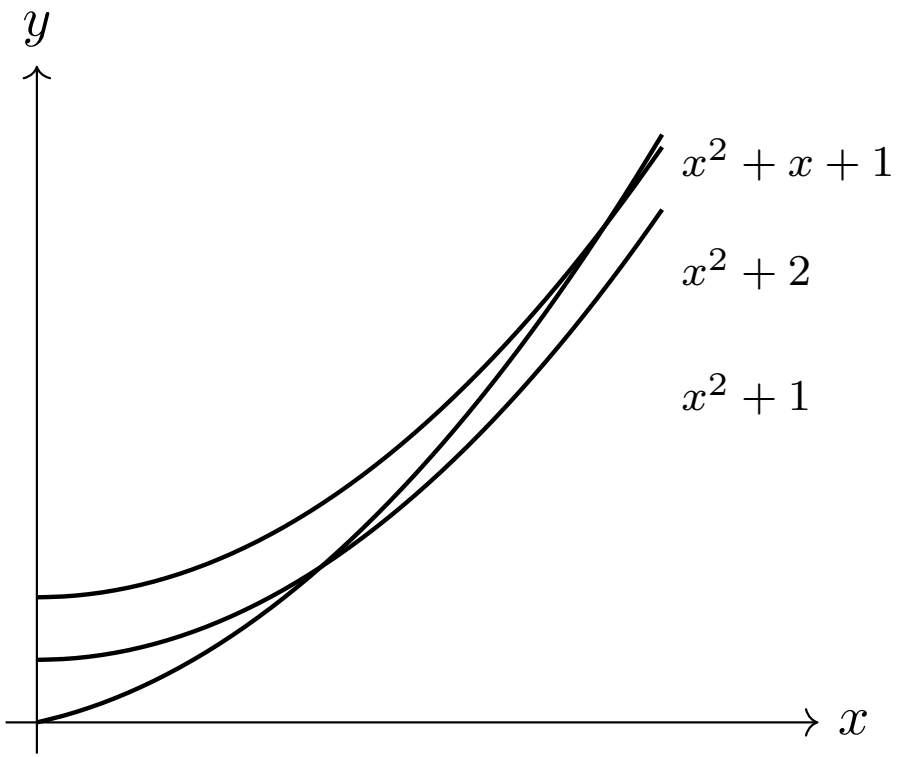
Same Shape, Different Constants



Message

All three graphs are straight lines through the origin. For growth comparison we treat them as the **same type** of function (linear), ignoring constant factors like 3 or $\frac{1}{3}$.

Parabolas with the Same Growth



Message

Different lower-order terms (like x or constants) shift the curves a bit, but for large x they all grow like x^2. So we say their growth is **quadratic**.

Big-O Notation

Definition (Big-O)

Let f and g be functions from the integers or real numbers to the real numbers. We say that $f(x)$ is $O(g(x))$ if there exist positive constants C and k such that

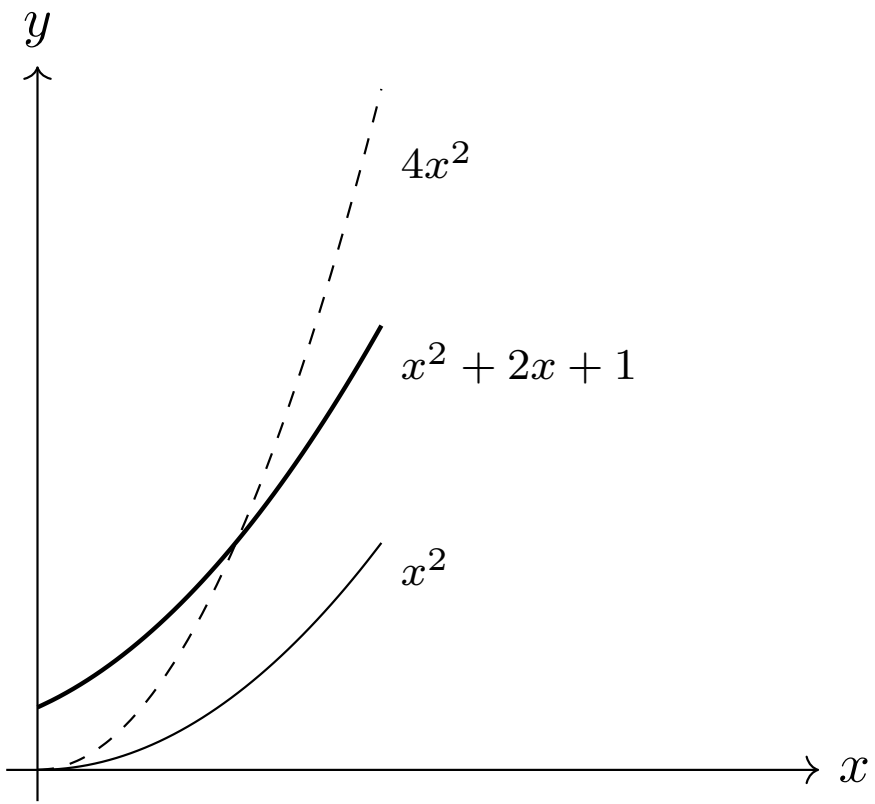
$$|f(x)| \leq C\,|g(x)| \quad \text{whenever } x > k.$$

We read this as: "$f(x)$ *is big-O of* $g(x)$".

Intuition

For large inputs, $f(x)$ does not grow faster than some constant multiple of $g(x)$. So $g(x)$ is an **upper bound** (up to a constant) on the growth of $f(x)$.

Big-O: Geometric Picture



Interpretation

For sufficiently large x, the graph of $f(x) = x^2 + 2x + 1$ lies **below** the dashed curve $4x^2$. So we can take $g(x) = x^2$, $C = 4$, and some k, and conclude

$$x^2 + 2x + 1 \text{ is } O(x^2).$$

Example: Showing $x^2 + 2x + 1$ is $O(x^2)$

Algebraic proof

For $x > 1$,

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2.$$

So $|f(x)| \leq 4|x^2|$ whenever $x > 1$. Thus we can choose $C = 4$ and $k = 1$, and by the definition:

$$x^2 + 2x + 1 \text{ is } O(x^2).$$

Message

To prove $f(x)$ is $O(g(x))$, it is enough to find **one** pair of constants C and k that works.

Big-O for Polynomials

Theorem 1

Let

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

where $a_0, a_1, \ldots, a_n$ are real numbers and $a_n \neq 0$. Then $f(x)$ is $O(x^n)$.

Intuition

For large x,

- the leading term $a_n x^n$ dominates the size of the polynomial;
- all lower-degree terms together can be bounded by some constant multiple of x^n.

So a polynomial of degree n has **growth rate** x^n.

Some Useful Big-O Estimates

Examples

For n a positive integer:

- $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$ is $O(n^2)$.
- $n! = 1 \cdot 2 \cdot 3 \cdots n$ grows **faster** than any polynomial; we write $n!$ is $O(n!)$ (trivial) and often compare it with 2^n, n^n, etc.
- $\log(n!)$ is $O(n \log n)$ (used for analysing some sorting algorithms).

Why these matter

These estimates appear again when we count the number of steps of algorithms like:

- insertion sort, merge sort, quicksort,
- tree operations, divide-and-conquer algorithms.

Upper Bound: $\log(n!) \in O(n \log n)$

Claim: $\log(n!) \in O(n \log n)$

For every $n \geq 1$, each factor of $n!$ satisfies $k \leq n$ for $1 \leq k \leq n$. Hence,

$$\begin{aligned} n! &= 1 \cdot 2 \cdot \ldots \cdot n \\ &\leq \underbrace{n \cdot n \cdot \ldots \cdot n}_{n \text{ times}} \\ &= n^n \end{aligned}$$

Taking logarithms:

$$n! \leq n^n \;\Rightarrow\; \log(n!) \leq \log(n^n) = n \log n$$

By definition of Big-O, choose $k = 1$ and $n_0 = 1$. Therefore,

$$\boxed{\log(n!) \in O(n \log n)}$$

Combining Big-O Estimates

Theorem 2 (Sum)

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$, then

$$f_1(x) + f_2(x) \text{ is } O\big(\max(|g_1(x)|, |g_2(x)|)\big).$$

So the **faster-growing** function dominates.

Theorem 3 (Product)

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$, then

$$f_1(x)\, f_2(x) \text{ is } O\big(g_1(x)\, g_2(x)\big).$$

Corollary 1

If both $f_1(x)$ and $f_2(x)$ are $O(g(x))$, then

$$f_1(x) + f_2(x) \text{ is also } O(g(x)).$$

Example 1: Big-O Estimate

Problem

$$f(n) = 3n \log(n!) + (n^2 + 3) \log n$$

Step 1: First Term

We know:

$$\log(n!) \in O(n \log n)$$

Using Product Rule:

$$3n \log(n!) = O(1) \cdot O(n) \cdot O(n \log n) = O(n^2 \log n)$$

$$\boxed{3n \log(n!) \in O(n^2 \log n)}$$

Example 1: Big-O Estimate (cont.)

Step 2: Second Term

$$(n^2 + 3) \log n$$

Since:

$$n^2 + 3 \in O(n^2) \quad \text{and} \quad \log n \in O(\log n)$$

Product Rule:

$$O(n^2) \cdot O(\log n) = O(n^2 \log n)$$

$$\boxed{(n^2 + 3) \log n \in O(n^2 \log n)}$$

Final Step: Sum Rule

$$O(n^2 \log n) + O(n^2 \log n) = O(n^2 \log n)$$

$$\boxed{f(n) \in O(n^2 \log n)}$$

Example 2: Big-O Estimate

Problem

$$f(x) = (x+1)\log(x^2+1) + 3x^2$$

Step 1: First Term

For large x:

$$x + 1 \in O(x)$$

Also:

$$x^2 + 1 \leq 2x^2 \Rightarrow \log(x^2+1) \in O(\log x^2)$$

Using log rule:

$$\log x^2 = 2\log x \Rightarrow \log(x^2+1) \in O(\log x)$$

Product Rule:

$$O(x) \cdot O(\log x) = O(x \log x)$$

$$\boxed{(x+1)\log(x^2+1) \in O(x\log x)}$$

Example 2: Big-O Estimate (cont.)

Step 2: Second Term

$$3x^2 \in O(x^2)$$

Final Step: Sum Rule

$$O(x \log x) + O(x^2)$$

Since x^2 grows faster than $x \log x$,

$$\boxed{f(x) \in O(x^2)}$$

Example 3: Big-O Estimate

Problem

$$f(n) = (3^n + n^5)(2^n + n^2)$$

Step 1: Simplify Each Factor

First factor:

$$3^n + n^5$$

Since exponential growth dominates polynomial growth,

$$3^n + n^5 \in O(3^n)$$

Second factor:

$$2^n + n^2$$

Since exponential growth dominates polynomial growth,

$$2^n + n^2 \in O(2^n)$$

$$\boxed{(3^n + n^5) \in O(3^n), \quad (2^n + n^2) \in O(2^n)}$$

Example 3: Big-O Estimate (cont.)

Step 2: Apply Product Rule

$$(3^n + n^5)(2^n + n^2)$$

Using:

$$O(3^n) \cdot O(2^n)$$

Product Rule:

$$O(3^n) \cdot O(2^n) = O(3^n 2^n)$$

Using exponent rule:

$$3^n 2^n = (3 \cdot 2)^n = 6^n$$

$$\boxed{f(n) \in O(6^n)}$$

Big-Omega Notation: Lower Bounds

Definition (Big-Omega)

Let f and g be functions. We say $f(x)$ is $\Omega(g(x))$ if there exist positive constants C and k such that

$$|f(x)| \geq C\,|g(x)| \quad \text{whenever } x > k.$$

We read this as: "$f(x)$ *is big-Omega of* $g(x)$".

Meaning

Big-Omega gives an **asymptotic lower bound**: for large inputs, $f(x)$ does not grow slower than a constant multiple of $g(x)$.

Example 1: Big-Omega Estimate

Problem

Show that

$$f(n) = 4n^3 + 2n^2 + 7 \in \Omega(n^3).$$

Step 1: Since all terms are positive and $4n^3$ dominates lower-degree terms,

$$4n^3 + 2n^2 + 7 \geq 4n^3 \quad \text{for all } n \geq 1.$$

Step 2: Choose constants

$$f(n) \geq 4n^3 = Cn^3$$

Choose $C = 4$ and $k = 1$.

$$\boxed{f(n) \in \Omega(n^3)}$$

Example 2: Big-Omega Estimate (cont.)

Problem

Show that

$$f(n) = 5n^4 - 3n^3 + 2n \in \Omega(n^4).$$

Step 1: For large n, the dominant term is $5n^4$. Observe that for $n \geq 1$:

$$-3n^3 + 2n \geq -3n^3$$

So,

$$f(n) \geq 5n^4 - 3n^3.$$

Step 2: Factor dominant term

$$5n^4 - 3n^3 = n^4\left(5 - \frac{3}{n}\right).$$

For $n \geq 2$:

$$5 - \frac{3}{n} \geq 5 - \frac{3}{2} = \frac{7}{2}. \text{ Thus, } f(n) \geq \frac{7}{2}n^4 \quad \text{for } n \geq 2.$$

Choose $C = \frac{7}{2}$ and $k = 2$.

$$\boxed{f(n) \in \Omega(n^4)}$$

Big-Theta Notation: Tight Bounds

Definition (Big-Theta)

We say $f(x)$ is $\Theta(g(x))$ if there exist positive constants C_1, C_2, and k such that

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)| \quad \text{whenever } x > k.$$

Interpretation

Big-Theta means:

- f is $O(g)$ (upper bound), and
- f is $\Omega(g)$ (lower bound).

So $g(x)$ describes the **exact order of growth** of $f(x)$.

Example: Big-Theta Estimate

Problem

Show that

$$f(n) = 6n^3 - 4n^2 + 9n$$

is $\Theta(n^3)$.

Upper Bound (Big-O) For $n \geq 1$:

$$6n^3 - 4n^2 + 9n \leq 6n^3 + 9n \leq 6n^3 + 9n^3 = 15n^3$$

$$\Rightarrow \quad f(n) \in O(n^3)$$

Example: Big-Theta Estimate (cont.)

Lower Bound (Big-Omega) Factor n^3:

$$f(n) = n^3 \left(6 - \frac{4}{n} + \frac{9}{n^2}\right)$$

For $n \geq 2$:

$$6 - \frac{4}{n} + \frac{9}{n^2} \geq 6 - 2 = 4$$

$$\Rightarrow \quad f(n) \geq 4n^3$$

$$\Rightarrow \quad f(n) \in \Omega(n^3)$$

Since $f(n)$ is both $O(n^3)$ and $\Omega(n^3)$,

$$\boxed{f(n) \in \Theta(n^3)}$$

What Is Time Complexity?

Idea

The **time complexity** of an algorithm is an estimate of how much time it needs to solve a problem of a given size.

How we measure it

Instead of seconds, we usually count a basic operation, for example:

- number of comparisons,
- number of assignments,
- number of loop iterations.

Time complexity is then written as a function of the input size n.

Example

If an algorithm uses about $5n + 3$ comparisons on inputs of size n, we say its time complexity is $\Theta(n)$ (linear time).

Worst-Case and Average-Case

Worst-case complexity

- Look at the **largest** number of operations the algorithm may use on any input of size n.
- Answers the question: "How bad can it get?"

Average-case complexity

- Assume all inputs of size n are equally likely (or use a suitable probability model).
- Compute the **average** number of operations over all such inputs.

Why both?

Worst-case gives a safe upper bound. Average-case can describe typical behaviour when worst-case is rare.

Example 1: Time Complexity of Maximum element Algorithm

Algorithm 1: Finding the Maximum Element

procedure max($a_1, a_2, \ldots, a_n$: integers)
  max := a_1
  for $i := 2$ **to** n
    if max $< a_i$ **then** max := a_i
  {max is the largest element}

Counting comparisons

1. The **for**-loop runs from $i = 2$ to $n \Rightarrow n - 1$ iterations.
2. In each iteration we do:
   - one comparison for the loop test $i \leq n$,
   - one comparison for the **if** test max $< a_i$.

   So there are $2(n - 1)$ comparisons inside the loop.
3. After the last iteration, there is one extra comparison $i \leq n$ to exit the loop.

Example 1: Time Complexity of Maximum element Algorithm

Conclusion

Total comparisons:

$$2(n-1)+1 = 2n-2+1 \quad \Rightarrow \quad \Theta(n).$$

So the algorithm uses exactly $2n-1$ comparisons, which leads to a time complecity of

$$\Theta(n).$$

Side note: Only n (input size) matters, not the actual values.

Example 2: Linear Search (Worst Case)

Algorithm 2: Linear Search

procedure linear_search(x : integer; $a_1, \ldots, a_n$)
  $i := 1$
  while $i \leq n$ **and** $x \neq a_i$ **do**
    $i := i + 1$
  if $i \leq n$ **then return** i
  else return 0 {0 means "not found"}

Assumption

Worst case: x is **not** in the list, so the loop runs for $i = 1, 2, \ldots, n$.

Counting comparisons

1. In each loop iteration:
   - comparison $i \leq n$,
   - comparison $x \neq a_i$.

   $\Rightarrow 2n$ comparisons while scanning the list.

Algorithm 2: Linear Search (cont...)

Counting comparisons

2. After the loop finishes, we evaluate:
   - $i \leq n$ in the final **if**,
   - then choose between `return i` and `return 0`.

   $\Rightarrow 2$ more comparisons.

Conclusion

Total comparisons:

$$2n + 2.$$

As n grows, this behaves like a constant multiple of n, so the worst-case time complexity is $\Theta(n)$.

Example 3: Binary Search (Worst Case)

Algorithm 3: Binary Search (sorted list)

procedure binary_search$(x; a_1, \ldots, a_n)$
  low := 1; high := n
  while low $\leq high$ **do**
    mid := $\lfloor (low + high)/2 \rfloor$
    if $x = a_{\text{mid}}$ **then return** mid
    else if $x < a_{\text{mid}}$ **then** high := mid - 1
    else low := mid + 1
  return 0 {0 means "not found"}

How the range shrinks

- Start: up to n elements in the search range.
- After 1 iteration: at most $n/2 = n/2^1$ elements left.
- After 2 iterations: at most $n/4 = n/2^2$ elements left.
- After k iterations: at most $n/2^k$ elements left.

We stop when $n/2^k \leq 1$.

Example 3: Step-by-Step Analysis (Worst Case)

Solving for k

$$\frac{n}{2^k} \leq 1 \quad \Rightarrow \quad 2^k \geq n \quad \Rightarrow \quad k \geq \log_2 n.$$

So the number of iterations (and comparisons) is about $\log_2 n$.

Conclusion

Binary search has worst-case time complexity

$$\Theta(\log n),$$

which grows much more slowly than the $\Theta(n)$ time of linear search.

Example 4: Linear Search (Average Case)

Problem

Describe the **average-case** performance of the linear search algorithm in terms of the average number of comparisons used, assuming:

- x is in the list $a_1, \ldots, a_n$,
- x is equally likely to be in any of the n positions.

Algorithm 2: Linear Search (again)

procedure linear_search(x : integer; $a_1, \ldots, a_n$)
  $i := 1$
  while $i \leq n$ **and** $x \neq a_i$ **do**
    $i := i + 1$
  if $i \leq n$ **then return** i
  else return 0

Example 4: Step-by-Step Average-Case Analysis

Goal

Find the **average** number of comparisons this algorithm makes, over all n possible positions of x.

Step 1: Comparisons if x is at position i

When x is at a_i $(1 \leq i \leq n)$:

- Inside the loop, for each of the i steps we check $i \leq n$ and $x \neq a_i$.
- When we finally find $x = a_i$, we exit the loop and do one more comparison $i \leq n$ in the final **if**.

So the total number of comparisons is of the form

$$\text{comparisons when } x \text{ is at } a_i = 2i + 1.$$

(You do 2 comparisons per step for i steps, plus 1 more outside.)

Example 4: Step-by-Step Average-Case Analysis

Step 2: Average over all positions

Each position i is equally likely, so the average number of comparisons is

$$\frac{(2 \cdot 1 + 1) + (2 \cdot 2 + 1) + \cdots + (2 \cdot n + 1)}{n} = \frac{3 + 5 + 7 + \cdots + (2n + 1)}{n}.$$

Factor out the common pattern:

$$3 + 5 + \cdots + (2n + 1) = 2(1 + 2 + \cdots + n) + n.$$

Using $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$, we get

$$\text{average} = \frac{2 \cdot \frac{n(n+1)}{2} + n}{n} = \frac{n(n+1) + n}{n} = n + 2.$$

So the average number of comparisons is $n + 2$. the average case time complexity is

$$\Theta(n),$$

Summary

Time complexity of our Algorithms

Algorithm	**Worst-case time**	**Main idea**
Max of n numbers	$\Theta(n)$	Scan once, keep current max
Linear search	$\Theta(n)$	Check elements one by one
Binary search (sorted)	$\Theta(\log n)$	Halve the search range each step
Linear search (average)	$\Theta(n)$	x equally likely in any position

Understanding the Complexity of Algorithms

Complexity	**Terminology**
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^k)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity